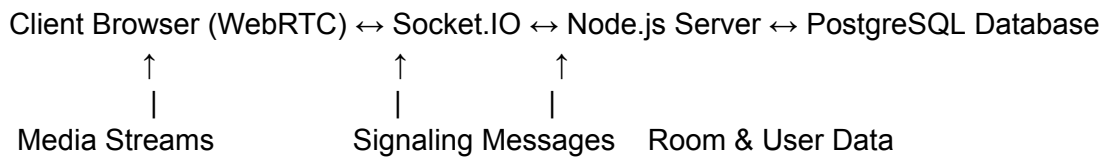




Architecture Overview

System Architecture Diagram



Data Flow

Client Connection: Browser connects to Node.js server via Socket.IO

Room Management: Client creates/joins rooms through signaling

WebRTC Negotiation: Peer-to-peer connections established via signaling

Media Exchange: Direct browser-to-browser media streaming

Data Persistence: Room and recording metadata stored in PostgreSQL



Technology Stack

Frontend

HTML5/CSS3/JavaScript: Core web technologies

WebRTC API: Real-time communication

Socket.IO Client: Real-time signaling

MediaRecorder API: Audio recording

Backend

Node.js: Runtime environment

Express.js: Web server framework

Socket.IO: Real-time bidirectional communication

PostgreSQL: Relational database

Multer: File upload handling

Infrastructure

PM2: Process manager for production

Nginx: Reverse proxy (optional)

PostgreSQL: Database server

Socket.IO Events

Client to Server Events

create-or-join-room: Create or join a room
user-data: Send user metadata
offer: Send WebRTC offer
answer: Send WebRTC answer
ice-candidate: Send ICE candidate
start-recording: Start recording session
stop-recording: Stop recording session
leave: Leave current room

Server to Client Events

created: Room created confirmation
joined: Room joined confirmation
user-joined: New user joined notification
user-left: User left notification
participants-updated: Participants list updated
user-updated: User data updated
offer: Receive WebRTC offer
answer: Receive WebRTC answer
ice-candidate: Receive ICE candidate
recording-started: Recording started confirmation
recording-stopped: Recording stopped confirmation
room-full: Room capacity reached
error: Error notification

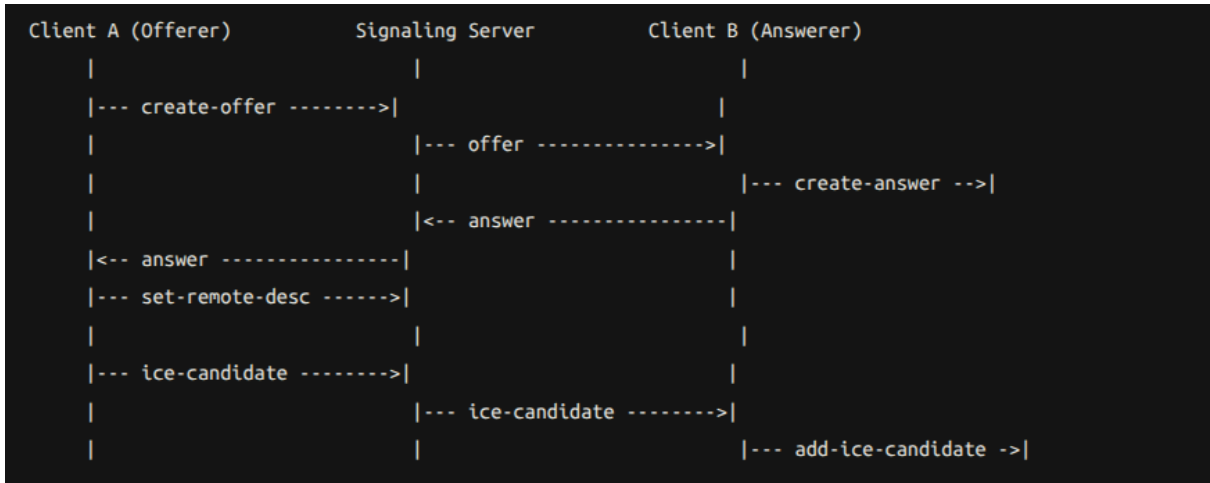


Signaling Process

Room Join Sequence

Client: create-or-join-room event with room name
Server: Validate room capacity, add user to room
Server: created or joined event response
Server: participants-updated event to all room members
Client: Create peer connections for existing participants
WebRTC: Offer/answer exchange through signaling
WebRTC: ICE candidate exchange
WebRTC: Peer connection established

WebRTC Negotiation Flow

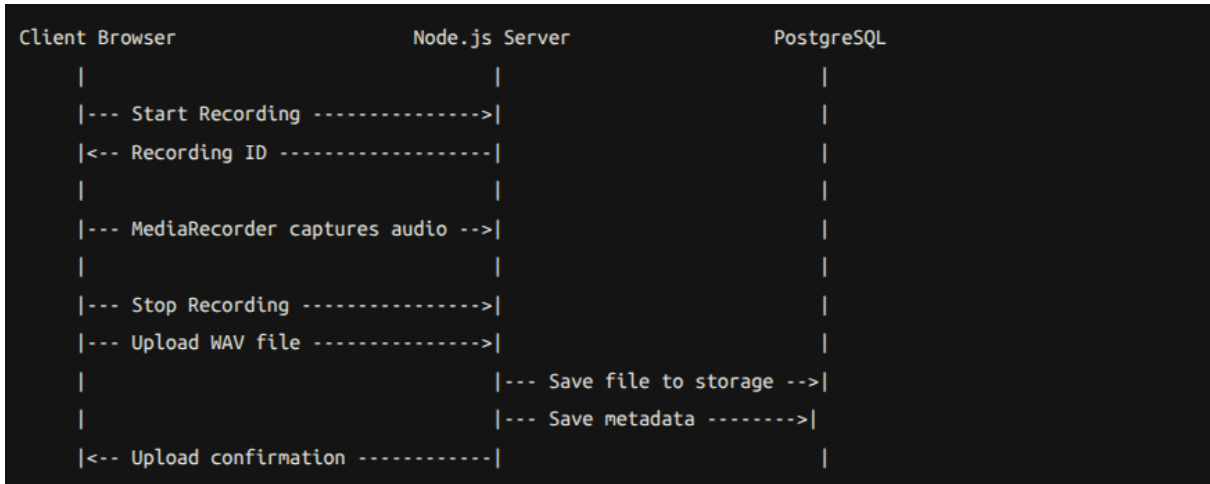


Recording System

Audio Recording Process

- Client:** MediaRecorder API captures audio stream
- Client:** Audio data collected in chunks
- Client:** Recording stopped, chunks combined into WAV file
- Client:** File uploaded to server via POST request
- Server:** File stored on disk, metadata saved to database
- Server:** Recording available for download via API

Recording API Flow



Quick Start Guide

1. Basic Usage (End Users)

Join a Meeting:
Visit <https://web.chatmybot.in>

Click "Start Media" to enable camera/microphone

Enter room name and click "Create" or "Join"

Share the room name with participants

Direct Room Access:

https://web.chatmybot.in?room=ROOM_NAME

Create Instant Meeting:

<https://web.chatmybot.in/new-meeting>

2. Features

- ✔ Real-time video/audio communication
- ✔ Screen sharing
- ✔ Recording functionality
- ✔ Up to 5 participants per room
- ✔ No authentication required
- ✔ Mobile responsive
- ✔ Cross-browser support

API Documentation

Base URL: <https://web.chatmybot.in/api>

1. Room Management APIs

Create Room

```
POST /api/rooms/create
Content-Type: application/json

{
  "room_name": "project-meeting",
  "max_participants": 5
}
```

Response:

```
{
  "success": true,
  "room": {
    "room_id": "project-meeting",
    "join_url": "https://web.chatmybot.in?room=project-meeting",
    "max_participants": 5
  }
}
```

Get Room Information

GET /api/room/project-meeting

Quick Create Room

GET /api/rooms/quick-create?name=quick-meeting

2. Recording APIs

Get Room Recordings

GET /api/recordings/room/project-meeting

Download Recording

GET /api/recordings/download/{recording_id}

Get Recording Details

GET /api/recordings/{recording_id}

3. System APIs

GET /health

System Statistics

GET /api/stats



Integration Guides

1. Node.js Application Integration

Installation:

```
npm install axios socket.io-client
```

Basic Integration:

```
-----  
const axios = require('axios');  
const io = require('socket.io-client');  
  
const API_BASE = 'https://web.chatmybot.in/api';  
  
class WebRTCIntegration {  
  constructor() {  
    this.socket = null;  
  }  
  
  // Create room  
  async createRoom(roomName, maxParticipants = 5) {  
    try {  
      const response = await axios.post(`${API_BASE}/rooms/create`, {  
        room_name: roomName,  
        max_participants: maxParticipants  
      });  
      return response.data;  
    } catch (error) {
```

```

        console.error('Error creating room:', error);
        throw error;
    }
}

// Join room via WebSocket
joinRoom(roomName, userData) {
    this.socket = io('https://web.chatmybot.in');

    this.socket.on('connect', () => {
        this.socket.emit('create-or-join-room', roomName, userData);
    });

    // Handle events
    this.socket.on('user-joined', (data) => {
        console.log('User joined:', data);
    });

    this.socket.on('offer', (data) => {
        // Handle WebRTC offer
    });

    return this.socket;
}

// Get recordings
async getRecordings(roomName) {
    try {
        const response = await axios.get(`${API_BASE}/recordings/room/${roomName}`);
        return response.data;
    } catch (error) {
        console.error('Error getting recordings:', error);
        throw error;
    }
}
}

module.exports = WebRTCIntegration;

```

Usage Example:

```

const WebRTCIntegration = require('./webrtc-integration');
const webrtc = new WebRTCIntegration();

// Create room
const room = await webrtc.createRoom('project-review', 5);

```

```
console.log('Room created:', room.join_url);

// Join room
const socket = webrtc.joinRoom('project-review', {
  name: 'John Doe',
  isVideoActive: true,
  isAudioActive: true
});
```

2. Angular Application Integration

Installation:

```
npm install axios socket.io-client
```

Service Implementation:

```
-----
// webrtc.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { io, Socket } from 'socket.io-client';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class WebRTCService {
  private apiBase = 'https://web.chatmybot.in/api';
  private socket: Socket;

  constructor(private http: HttpClient) {}

  // Create room
  createRoom(roomName: string, maxParticipants: number = 5): Observable<any> {
    return this.http.post(`${this.apiBase}/rooms/create`, {
      room_name: roomName,
      max_participants: maxParticipants
    });
  }

  // Join room
  joinRoom(roomName: string, userData: any): Socket {
    this.socket = io('https://web.chatmybot.in');

    this.socket.on('connect', () => {
```



```

    this.socket.emit('create-or-join-room', roomName, userData);
  });

  return this.socket;
}

// Get recordings
getRecordings(roomName: string): Observable<any> {
  return this.http.get(`${this.apiBase}/recordings/room/${roomName}`);
}

// Download recording
downloadRecording(recordingId: string): void {
  window.open(`${this.apiBase}/recordings/download/${recordingId}`, '_blank');
}
}

```

Component Usage:

```

// video-room.component.ts
import { Component, OnInit, OnDestroy } from '@angular/core';
import { WebRTCService } from '../services/webrtc.service';

@Component({
  selector: 'app-video-room',
  templateUrl: './video-room.component.html',
  styleUrls: ['./video-room.component.css']
})
export class VideoRoomComponent implements OnInit, OnDestroy {
  roomName: string = '';
  localStream: MediaStream;
  remoteStreams: Map<string, MediaStream> = new Map();

  constructor(private webrtcService: WebRTCService) {}

  async ngOnInit() {
    // Auto-join if room parameter exists
    const urlParams = new URLSearchParams(window.location.search);
    const roomId = urlParams.get('room');

    if (roomId) {
      this.joinRoom(roomId);
    }
  }
}

```

```

}

async joinRoom(roomName: string) {
  try {
    // Get user media
    this.localStream = await navigator.mediaDevices.getUserMedia({
      video: true,
      audio: true
    });

    // Join room
    const socket = this.webrtcService.joinRoom(roomName, {
      name: 'User',
      isVideoActive: true,
      isAudioActive: true
    });

    // Handle WebRTC events
    socket.on('offer', this.handleOffer.bind(this));
    socket.on('answer', this.handleAnswer.bind(this));
    socket.on('ice-candidate', this.handleIceCandidate.bind(this));

  } catch (error) {
    console.error('Error joining room:', error);
  }
}

private handleOffer(data: any) {
  // Handle WebRTC offer
}

private handleAnswer(data: any) {
  // Handle WebRTC answer
}

private handleIceCandidate(data: any) {
  // Handle ICE candidate
}

ngOnDestroy() {
  if (this.localStream) {
    this.localStream.getTracks().forEach(track => track.stop());
  }
}
}

```

3. Java Spring Boot Integration

Dependencies (pom.xml):

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
  </dependency>
  <dependency>
    <groupId>com.squareup.okhttp3</groupId>
    <artifactId>okhttp</artifactId>
    <version>4.9.3</version>
  </dependency>
</dependencies>
```

Service Implementation:

```
// WebRTCService.java
@Service
public class WebRTCService {
    private static final String API_BASE = "https://web.chatmybot.in/api";
    private final OkHttpClient client = new OkHttpClient();

    public String createRoom(String roomName, int maxParticipants) throws IOException {
        String json = String.format("{\"room_name\":\"%s\",\"max_participants\":%d",
            roomName, maxParticipants);

        Request request = new Request.Builder()
            .url(API_BASE + "/rooms/create")
```

```

        .post(RequestBody.create(json, MediaType.get("application/json")))
        .build();

    try (Response response = client.newCall(request).execute()) {
        if (!response.isSuccessful()) throw new IOException("Unexpected code " + response);
        return response.body().string();
    }
}

public String getRoomInfo(String roomName) throws IOException {
    Request request = new Request.Builder()
        .url(API_BASE + "/room/" + roomName)
        .build();

    try (Response response = client.newCall(request).execute()) {
        return response.body().string();
    }
}

public String getRecordings(String roomName) throws IOException {
    Request request = new Request.Builder()
        .url(API_BASE + "/recordings/room/" + roomName)
        .build();

    try (Response response = client.newCall(request).execute()) {
        return response.body().string();
    }
}
}

```

Controller:

```

// VideoConferenceController.java
@RestController
@RequestMapping("/api/video")
public class VideoConferenceController {

    @Autowired
    private WebRTCService webrtcService;

    @PostMapping("/create-room")
    public ResponseEntity<String> createRoom(@RequestParam String roomName,
                                             @RequestParam(defaultValue = "5") int maxParticipants) {
        try {

```

```

        String result = webrtcService.createRoom(roomName, maxParticipants);
        return ResponseEntity.ok(result);
    } catch (IOException e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Error creating room: " + e.getMessage());
    }
}

@GetMapping("/recordings/{roomName}")
public ResponseEntity<String> getRecordings(@PathVariable String roomName) {
    try {
        String result = webrtcService.getRecordings(roomName);
        return ResponseEntity.ok(result);
    } catch (IOException e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Error getting recordings: " + e.getMessage());
    }
}
}

```



Deployment Guide

1. Server Requirements

Node.js: 16.x or higher

PostgreSQL: 12.x or higher

PM2: For process management

Nginx: Reverse proxy (optional)

SSL Certificate: For HTTPS

Logging:

Application logs: `pm2 logs webrtc-video-app`

Error logs: `/root/.pm2/logs/webrtc-video-app-error.log`

Output logs: `/root/.pm2/logs/webrtc-video-app-out.log`



Support & Resources

Documentation:

WebRTC Documentation: <https://webrtc.org/>

Socket.IO Documentation: <https://socket.io/docs/>

PostgreSQL Documentation: <https://www.postgresql.org/docs/>

Security Considerations

Best Practices

HTTPS Enforcement: Always use HTTPS in production

Input Validation: Validate all user inputs

Authentication: Implement user authentication

Rate Limiting: Prevent abuse with rate limits

CORS Configuration: Restrict cross-origin requests

File Upload Security: Validate file types and sizes

Database Security

Use parameterized queries to prevent SQL injection

Limit database user privileges

Regularly update PostgreSQL security patches



Scaling Strategies

Vertical Scaling

Increase server resources (CPU, RAM)

Optimize database performance with indexes

Implement connection pooling

Database Scaling

Implement read replicas for PostgreSQL

Use connection pooling with PgBouncer

Optimize queries with proper indexing

Media Scaling

Implement SFU (Selective Forwarding Unit) for large meetings

Use dedicated TURN servers for NAT traversal

Consider commercial WebRTC platforms for enterprise scale

